

Week 5

Semantics

Guan-Yun Wang

Department of Information Management

Fu Jen Catholic University

2026.03.27

Representing Text - Capturing Semantics

- Representing the meaning of words, phrases, and sentences in a form that's understandable to computers is one of the pillars of NLP processing.
- Machine learning represents each data point as a list of numbers (a fixed-size vector).
 - Engineer: how to turn words and sentences into these vectors.
 - vector -> numeric form

The topics:

- Creating a simple classifier
- Putting documents into a bag of words
- Constructing an N-gram model
- Representing texts with TF-IDF
- Using word embeddings
- Training your own embeddings model
- Using MistralAI embeddings instead of word embeddings

Technical requirements

Download the model and datasets.

- The Google `word2vec` model:
<https://drive.google.com/file/d/0B7XkCwpI5KDYNINUTTISS21pQmM/edit?resourcekey=0-wjGZdNAUop6WykTtMip30g>
- The IMDB movie dataset: <https://github.com/venusarvi/imdb-movies/blob/main/IMDB-Movie-Data.csv>

Scikit-learn

If you never access any machine learning model in python, please install scikit-learn in your environment.

<https://scikit-learn.org/stable/install.html>

```
pip install -U scikit-learn
```

Installing the latest release

Operating System

Windows

MacOS

Linux

Package Manager

pip

conda

Install the 64-bit version of Python 3, for instance from the [official website](#).

Now create a [virtual environment \(venv\)](#) and install scikit-learn. Note that the virtual environment is optional but strongly recommended, in order to avoid potential conflicts with other packages.

```
PS C:\> python -m venv sklearn-env
PS C:\> sklearn-env\Scripts\activate # activate
PS C:\> pip install -U scikit-learn
```

Creating a simple classifier

- The reason why we need to represent text as vectors is to make it into a computer-readable form.
- **Computers can't understand words but are good at manipulating numbers**
- Here we're going to download a dataset from Hugging Face.
- We will create a classifier that will assign either negative or positive sentiment to *Rotten Tomatoes* review.

```
from datasets import load_dataset
```

<https://huggingface.co/docs/datasets/loading>

Before you use it you need to install Datasets (for example, with pip)

```
pip install datasets
```

Implementation

- Load the train and test datasets from Hugging Face.
- For both the training and test sets, we will select the first and last 15% of the data instead of loading the full datasets.

```
from datasets import load_dataset
train_dataset = load_dataset("rotten_tomatoes", split="train[:15%]+train[-15%:]")
test_dataset = load_dataset("rotten_tomatoes", split="test[:15%]+test[-15%:]")
```

- Check and print out the length of each dataset

```
print(len(train_dataset))
print(len(test_dataset))
```

pos vectorizer

- Create the `POS_vectorizer` class.
- The method `vectorize` processes the text and counts the number of verbs, nouns, proper nouns, adjectives, adverbs, auxiliary verbs, pronouns, numbers, and punctuation marks.
- By using a `spaCy` model, each piece of text is turned into a vector of size 10.
 - The first element of the vector is the length of the text, and the other numbers indicate the number of words in the text of that particular part of speech.

```
class POS_vectorizer:
    def __init__(self, spacy_model):
        self.model = spacy_model

    def vectorize(self, input_text):
        doc = self.model(input_text)
        vector = []
        vector.append(len(doc))
        pos = {"VERB":0, "NOUN":0, "PROPN":0, "ADJ":0, "ADV":0, "AUX":0, "PRON":0, "NUM":0, "PUNCT":0}
        for token in doc:
            if token.pos_ in pos.keys():
                pos[token.pos_] += 1
        vector_values = list(pos.values())
        vector = vector + vector_values
        return vector
```

- Remember to call the `spaCy` 's model

```
import spacy
small_model = spacy.load("en_core_web_sm")
```

- Test out the `POS_vectorizer` class.

```
sample_text = train_dataset[0]["text"]
vectorizer = POS_vectorizer(small_model)
vector = vectorizer.vectorize(sample_text)
```


The result should look like this.

```
print(sample_text)
print(vector)
```

```
the rock is destined to be the 21st century's new " conan " and that he's going
to make a splash even greater than arnold schwarzenegger , jean-claud van
damme or steven segal . [38, 3, 8, 3, 4, 1, 3, 1, 0, 5]
```

Prepare the data for training our classifier.

- Import `pandas` and the `numpy` packages and create two dataframes (one is for training and the other one for testing).
- In each dataset, we create a new column called `vector` that contains the vector for this piece of text.
- Use the `apply` method to turn the text into vectors and store them in the new column (`vector`) (by using a lambda function)
- Then, turn the vector and the label columns into `numpy` arrays to have the data in the right format for the classifier (`stack` is used for aligning the arrays)

```
import pandas as pd
import numpy as np
train_df = train_dataset.to_pandas()
train_df.sample(frac=1)
test_df = test_dataset.to_pandas()
train_df["vector"] = train_df["text"].apply(lambda x: vectorizer.vectorize(x))
test_df["vector"] = test_df["text"].apply(lambda x: vectorizer.vectorize(x))
X_train = np.stack(train_df["vector"].values, axis=0)
X_test = np.stack(test_df["vector"].values, axis=0)
y_train = train_df["label"].to_numpy()
y_test = test_df["label"].to_numpy()
```

Train the classifier

- Here we choose the logistic regression algorithm, since it is one of the simplest algorithms, as well as one of the fastest.

(Here you can find more models: https://scikit-learn.org/stable/api/sklearn.linear_model.html)

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
clf = LogisticRegression(C=0.1)
clf = clf.fit(X_train, y_train)
```

- After import the `LogisticRegression` class and `classification_report` method from `sklearn`, we create the `LogisticRegression` object.

We can test the classifier on the test data by applying the `predict` method to the vectors in the test data and print out the classification report.

```
test_df["prediction"] = test_df["vector"].apply(lambda x: clf.predict([x])[0])  
print(classification_report(test_df["label"], test_df["prediction"]))
```

Here we can see that the overall accuracy is low. (0.58, slightly above chance.) This is because the vector representation we used is very crude.

Putting documents into a bag of words

- A **bag of words** is the simplest way of representing a text.
- We treat our text as a collection of *documents*.
 - *Documents* can be anything from sentences to scientific articles to blog posts or whole books. (Depending on the engineering question.)
- The bag of words method uses a "training" text that provides it with a list of words that it should consider.
 - When encoding new sentences, it counts the number of occurrences each word makes in the document, and the final vector includes those counts for each word in the vocabulary.
 - It does not take into account the relationships of words between themselves and **only counts the number of occurrences of each word**.

Creating a bag of words for the Rotten Tomatoes reviews

Load the training and testing dataframes.

```
def load_train_test_dataset_pd():  
    train_dataset = load_dataset("rotten_tomatoes", split="train[:15%]+train[-15%:]")  
    test_dataset = load_dataset("rotten_tomatoes", split="test[:15%]+test[-15%:]")  
    train_df = train_dataset.to_pandas()  
    train_df.sample(frac=1)  
    test_df = test_dataset.to_pandas()  
    return (train_df, test_df)
```

```
(train_df, test_df) = load_train_test_dataset_pd()
```

Create the vectorizer, fit it on the training data, and print out the resulting matrix. We will use the `max_df` parameter to specify which words should be used as stop words.

- `max_df=0.4` means words that appear in more than 40% of the documents should be ignored when constructing the vectorizer.

```
vectorizer = CountVectorizer(max_df=0.4)
X = vectorizer.fit_transform(train_df["text"])
print(X)
```

The printout should like `(row, column) value`. It means (the document index, word index) frequency

Besides, you can use `max_df=300` to set an absolute threshold on the word frequency. This means we will use all words whose frequency is lower than 300 across documents.

```
vectorizer = CountVectorizer(max_df=300)
X = vectorizer.fit_transform(train_df["text"])
```

You can also provide a list of stop words to the vectorizer.

```
vectorizer = CountVectorizer(stop_words=['the', 'this', 'these', 'in', 'at', 'for'])
X = vectorizer.fit_transform(train_df["text"])
```


In most cases, we use a different format to represent vectors, a dense matrix that is easier to use in practice. Instead of specifying rows and columns with numbers, they are inferred from the position of the value.

```
dense_matrix = X.todense()  
print(dense_matrix)
```

Here we can see all the words used in the document set and the length of the vocabulary.

- This can be used as a sanity check and to see whether there are any irregularities in the vocabulary.

```
print(vectorizer.get_feature_names_out())  
print(len(vectorizer.get_feature_names_out()))
```

We can also see all the stop words used by the vectorizer

```
print(vectorizer.stop_words_)
```

If you set appear in more than 40% of reviews,
The stop words should be????_____

Test the effect of bag-of-words vectorizer

We can also use the `CountVectorizer` object to represent new reviews that were not in the original document set.

Here we test the effect of this bag-of-words vectorizer on the simple classifier

```
vectorizer = CountVectorizer(max_df=300)
(train_df, test_df) = load_train_test_dataset_pd()
X = vectorizer.fit_transform(train_df["text"])
vectorize = lambda x: vectorizer.transform([x]).toarray()[0]
(X_train, X_test, y_train, y_test) = create_train_test_data(train_df, test_df, vectorize)
clf = train_classifier(X_train, y_train)
test_classifier(test_df, clf)
```

Constructing an N-gram model

- Representing a document as a bag of words is useful, but semantics is about more than just words in isolation.
- To capture *word combinations*, an **n-gram model** is useful.
- Here we're going to build **bigram model**, here bigrams are sequences of two words.

Implementation

Here we also use `CountVectorizer` class. We will use the `ngram_range` argument, which counts not only individual words but also word combinations.

- `ngram_range` means the number of words in the combinations ranges from 1 to 2.

```
bigram_vectorizer = CountVectorizer(ngram_range=(1, 2), max_df=300)
X = bigram_vectorizer.fit_transform(train_df["text"])
```

```
print(bigram_vectorizer.get_feature_names_out())
print(len(bigram_vectorizer.get_feature_names_out()))
```

Here you will see the length of the vocabulary is much **larger/smaller** than the length of the unigram vectorizer.

Finally, we train a simple classifier using the new bigram vectorizer.

- The resulting accuracy is slightly worse than the accuracy of the classifier that uses a unigram vectorizer from the previous section.
- Therefore, actually you can try other n-gram range to test. Because the phrases are not always two words, there should have other possible word combinations.

```
vectorize = lambda x: bigram_vectorizer.transform([x]).toarray()[0]
(X_train, X_test, y_train, y_test) = create_train_test_data(train_df, test_df, vectorize)
clf = train_classifier(X_train, y_train)
test_classifier(test_df, clf)
```

Representing texts with TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

- Create the vectorizer and fit it on the training text. `max_df` also is used to exclude stop words.

```
vectorizer = TfidfVectorizer(max_df=300)  
vectorizer.fit(train_df["text"])
```

- Check if the data has anything wrong or not. Since here we use unigram, the size of the vocabulary should be the same as the one in the bag-of-words implementation

```
print(vectorizer.get_feature_names_out())  
print(len(vectorizer.get_feature_names_out()))
```

Train the classifier and see the score of the results.

```
vectorize = lambda x: vectorizer.transform([x]).toarray()[0]
(X_train, X_test, y_train, y_test) = create_train_test_data(train_df, test_df, vectorize)
clf = train_classifier(X_train, y_train)
test_classifier(test_df, clf)
```

How it work?

$$TF = \frac{\text{Number of times a term appears in the document}}{\text{Total number of words in the document}}$$

$$IDF = \frac{\text{Total number of documents}}{\text{Number of documents where term occurs}}$$

$$TF - IDF = TF * IDF$$

TF: term frequency; IDF: Inverse document frequency. TF is the number of times a word occurs in the document. IDF is the total number of documents divided by the number of documents where the word occurs.

There's more

We can build `TfidfVectorizer` and use **character n-gram** instead of **word n-grams**

```
tfidf_char_vectorizer = TfidfVectorizer(analyzer='char_wb', ngram_range=(1,5))  
tfidf_char_vectorizer = tfidf_char_vectorizer.fit(train_df["text"])
```

Using word embeddings

Getting ready

```
!pip install --upgrade gensim
```

Import the `gensim` package

```
import gensim
```

Here we're going to use a pretrained `word2vec` model. You can download the model online and have this file in your path. (`GoogleNews-vectors-negative300.bin.gz`)

```
model = gensim.models.KeyedVectors.load_word2vec_format(  
    '[YOUR PATH HERE]/data/GoogleNews-vectors-negative300.bin.gz',  
    binary=True)
```

Using the pretrained model, we can now load individual word vectors. Here, we load the word vector for the word *king*.

- The result is a long vector that represents this word in the `word2vec` model.

```
vec_king = model['king']  
print(vec_king)
```

We can also get words that are most similar to a given word. For example, let's print out the words most similar to *apple* and *tomato*.

- The score is the cosine distance between a pair of vectors. * The larger the score, the more similar the two words.

observe the results:

```
[('apples', 0.720359742641449), ('pear', 0.6450695991516113), ('fruit', 0.6410146951675415), ('berry',  
0.6302294731140137), ('pears', 0.6133962273597717), ('strawberry', 0.6058260798454285), ('peach',  
0.6025872230529785), ('potato', 0.5960934162139893), ('grape', 0.5935864448547363), ('blueberry',  
0.5866668224334717), ('cherries', 0.5784382820129395), ('mango', 0.5751855969429016), ('apricot',  
0.5727778673171997), ('melon', 0.5719985365867615), ('almond', 0.5704829692840576)]  
[('tomatoes', 0.8442262411117554), ('lettuce', 0.7069937586784363), ('asparagus', 0.7050933837890625),  
('peaches', 0.6938520073890686), ('cherry_tomatoes', 0.689752995967865), ('strawberry',  
0.6888598203659058), ('strawberries', 0.6832594871520996), ('bell_peppers', 0.6813561916351318), ('potato',  
0.6784172058105469), ('cantaloupe', 0.6780218482017517), ('celery', 0.6751954555511475), ('onion',  
0.6740139126777649), ('cucumbers', 0.6706331968307495), ('spinach', 0.6682619452476501), ('cauliflower',  
0.6681588292121887)]
```

Sentence Vector

We can compute a **sentence vector** by averaging all the word vectors in the sentence.

```
def get_word_vectors(sentence, model):  
    word_vectors = []  
    for word in sentence:  
        try:  
            word_vector = model[word.lower()]  
            word_vectors.append(word_vector)  
        except KeyError:  
            continue  
    return word_vectors
```

```
def get_sentence_vector(word_vectors):  
    matrix = np.array(word_vectors)  
    centroid = np.mean(matrix[:, :], axis=0)  
    return centroid
```

```
vectorize = lambda x: get_sentence_vector(get_word_vectors(x, model))
(train_df, test_df) = load_train_test_dataset_pd()
(X_train, X_test, y_train, y_test) = create_train_test_data(train_df, test_df, vectorize)
clf = train_classifier(X_train, y_train)
test_classifier(test_df, clf)
```

You may see that the scores are much lower than those in previous sections. There might be several reasons for this

- the `word2vec` model is English-only, but the data is multilingual.

Besides, `gensim` can do something fun.

For example, if you want to compile a list of words with one that doesn't match.

```
word = ['baanan', 'apple', 'computer', 'strawberry']  
print(model.doesnt_match(words))
```

You can also find a word that's most similar to another word.

```
word = "cup"  
words = ['glass', 'computer', 'pencil', 'watch']  
print(model.most_similar_to_given(word, words))
```

You also notice that the processing speed is quick.

Training your own embeddings model

It is possible to train our own `word2vec` model on a corpus.

We're going to use the Rotten Tomatoes reviews. But the dataset is not very large, the results are not as good as they could be with a larger collection.

- Import the packages and functions

```
import gensim
from gensim.models import Word2Vec
from datasets import load_dataset
from gensim import utils
```


- Load the training data and check its length

```
train_dataset = load_dataset("rotten_tomatoes", split="train")  
print(len(train_dataset))
```

- Create the `RottenTomatoesCorpus` class. The `word2vec` training algorithm requires a class with a defined `__iter__` function that allows you to iterate through the data, so that is why we need this class:

```
class RottenTomatoesCorpus:  
  
    def __init__(self, sentences):  
        self.sentences = sentences  
  
    def __iter__(self):  
        for review in self.sentences:  
            yield utils.simple_preprocess(gensim.parsing.preprocessing.remove_stopwords(review))
```

- Create an instance of `RottenTomatoesCorpus` using the loaded training dataset.

```
sentences = train_dataset["text"]  
corpus = RottenTomatoesCorpus(sentences)
```

- Initialize `word2vec` model, train it, and then save it to disk.

```
model = Word2Vec(sentences=corpus, vector_size=100, window=5, min_count=1, workers=4)  
model.train(corpus_iterable=corpus, total_examples=model.corpus_count, epochs=100)  
model.save("../data/rotten_tomato_word2vec.model")
```

`min_count` refers to the minimum number of times a word has to occur in the training corpus.

`size` sets the size of the word vector

`window` restricts the maximum number of words between the predicted and current words in a sentence.

`workers` refers to the number of working threads.

`epoch` the number of training iterations

Get 10 words similar to the word *movie*.

```
w1 = "movie"  
words = model.wv.most_similar(w1, topn=10)  
print(words)
```

Using Mistral AI embedding model

Text embedding model from Mistral AI:

https://docs.mistral.ai/capabilities/embeddings/text_embeddings

```
from mistralai.client import Mistral
mistral_client = Mistral(api_key=MISTRAL_API_KEY)
model = "mistral-embed"
```

```
text = "I love jazz"
response = mistral_client.embeddings.create(
    model=model,
    inputs=text
)
print(response)
embeddings = response.data[0].embedding
print(embeddings)
```

check the results

```
import time
vectorize = lambda x: get_sentence_vector(x, model)
(train_df, test_df) = load_train_test_dataset_pd()
start = time.time()
(X_train, X_test, y_train, y_test) = create_train_test_data(train_df, test_df, vectorize)
print(f"MistralAI embeddings: {time.time() - start} s")
clf = train_classifier(X_train, y_train)
test_classifier(test_df, clf)
```